Міністерство освіти і науки України

Львівський національний університет імені Івана Франка

Факультет прикладної математики та інформатики

Бази даних та інформаційні системи

Лабораторна робота №11

Тема: «Мова XML та її використання в СКБД PostgreSQL»

з курсу “Бази даних та інформаційні системи ”

Виконав:

Ст. Яценко Владислав Тарасович

Група: ПМі-33

Львів – 2025

Тема: Вивчення мови XML та її використання в СКБД PostgreSQL

Мета роботи: Ознайомлення з конструкціями мови XML та її використанням в СКБД PostgreSQL, зокрема, зі створенням XML даних та XML документів, перетворенням таблиць реляційної бази в XML-документ.

Хід роботи

Прочитав теоретичні відомості про мову XML, її переваги, елементи, теги, представлення даних, створення значень, обробку кодування, функції, створення контенту, умови, обробку значень, відображення таблиць.

У запиті використаний select з функціями створення xml контенту xmlroots, xmlattributes, xmlelement, xmlforest та xmlagg.

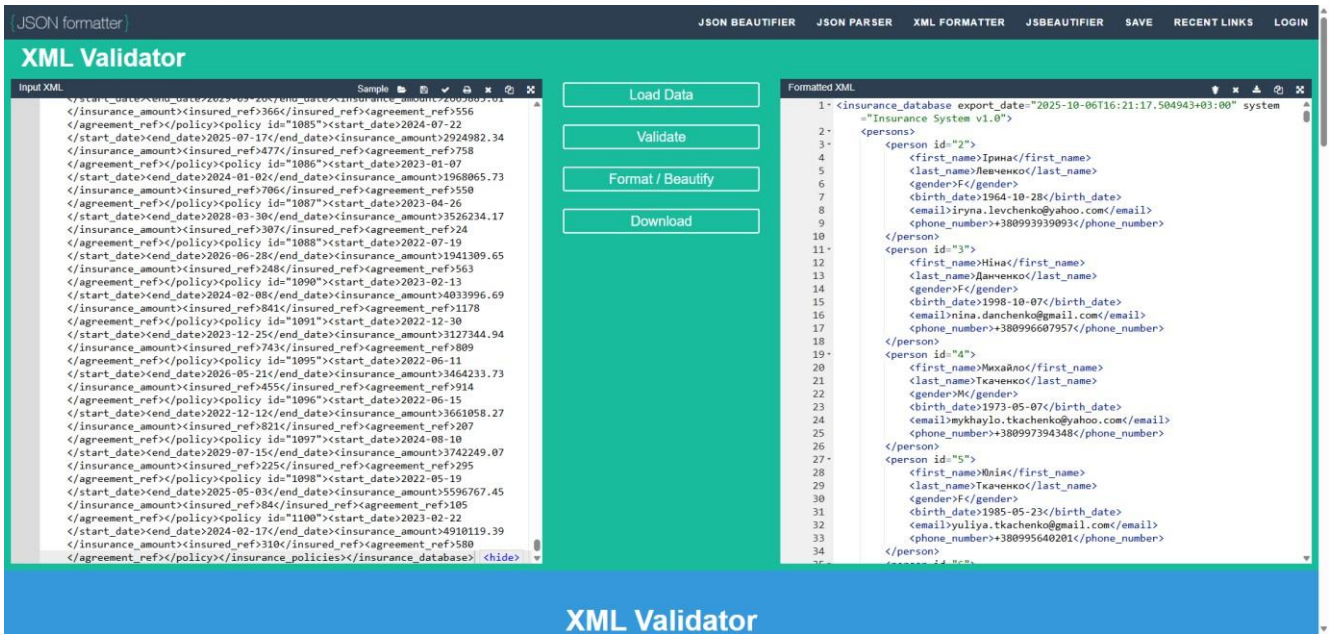
```
SELECT xmlroot(
    xmlelement(
        NAME "insurance_database",
        xmlattributes(
            CURRENT_TIMESTAMP AS "export_date",
            'Insurance' AS "system"
        ),

        xmlelement(NAME "persons",
            (SELECT xmlagg(
                xmlelement(NAME "person",
                    xmlattributes(PERSON_ID AS "id"),
                    xmlforest(
                        FIRST_NAME AS "first_name",
                        LAST_NAME AS "last_name",
                        GENDER AS "gender",
                        BIRTH_DATE AS "birth_date",
                        EMAIL AS "email",
                        PHONE_NUMBER AS "phone_number"
                    )
                )
            ) FROM PERSON_CHARACTERISTICS)
        ),

        xmlelement(NAME "properties",
            (SELECT xmlagg(
                xmlelement(NAME "property",
                    xmlattributes(
                        PROPERTY_ID AS "id",
                        CURRENCY AS "currency"
                    ),
                    xmlforest(
                        DESCRIPTION AS "description",
                        VALUE AS "value",
                        YEAR_OF_ISSUE AS "year_of_issue",
                        LOCATION AS "location"
                    )
                )
            ) FROM PROPERTY_CHARACTERISTICS)
        )
```

```
        xmlelement(NAME "agreements",
            (SELECT xmlagg(
                xmlelement(NAME "agreement",
                    xmlattributes(
                        AGREEMENT_ID AS "id",
                        STATUS AS "status"
                    ),
                    xmlforest(
                        CONCLUSION_DATE AS "conclusion_date",
                        TOTAL_INSURANCE_AMOUNT AS "total_insurance_amount",
                        DURATION AS "duration_months"
                    )
                )
            ) FROM AGREEMENT)
        ),

        xmlelement(NAME "insured_persons",
            (SELECT xmlagg(
                xmlelement(NAME "insured",
                    xmlattributes(
                        INSURED_ID AS "id",
                        PERSON_ID AS "person_ref"
                    ),
                    xmlforest(
                        ADDRESS AS "address",
                        PHONE_NUMBER AS "phone_number",
                        EMAIL AS "email",
                        GENDER AS "gender",
                        BIRTH_DATE AS "birth_date"
                    )
                )
            ) FROM INSURED)
        ),

        xmlelement(NAME "insurance_objects",
            (SELECT xmlagg(
                xmlelement(NAME "insurance_object",
                    xmlattributes(
                        OBJECT_ID AS "id",
                        OBJECT_TYPE AS "type"
                    ),
                    xmlforest(
                        INSURED_ID AS "insured_ref",
                        PROPERTY_ID AS "property_ref"
                    )
                )
            ) FROM INSURANCE_OBJECT)
        ),

        xmlelement(NAME "insurance_policies",
            (SELECT xmlagg(
                xmlelement(NAME "policy",
                    xmlattributes(POLICY_ID AS "id"),
                    xmlforest(
                        START_DATE AS "start_date",
                        END_DATE AS "end_date",
                        INSURANCE_AMOUNT AS "insurance_amount",
                        INSURED_ID AS "insured_ref",
                        AGREEMENT_ID AS "agreement_ref"
                    )
                )
            ) FROM INSURANCE_POLICY)
        )
    ),
    VERSION '1.0'
)::text AS xml_content;
```

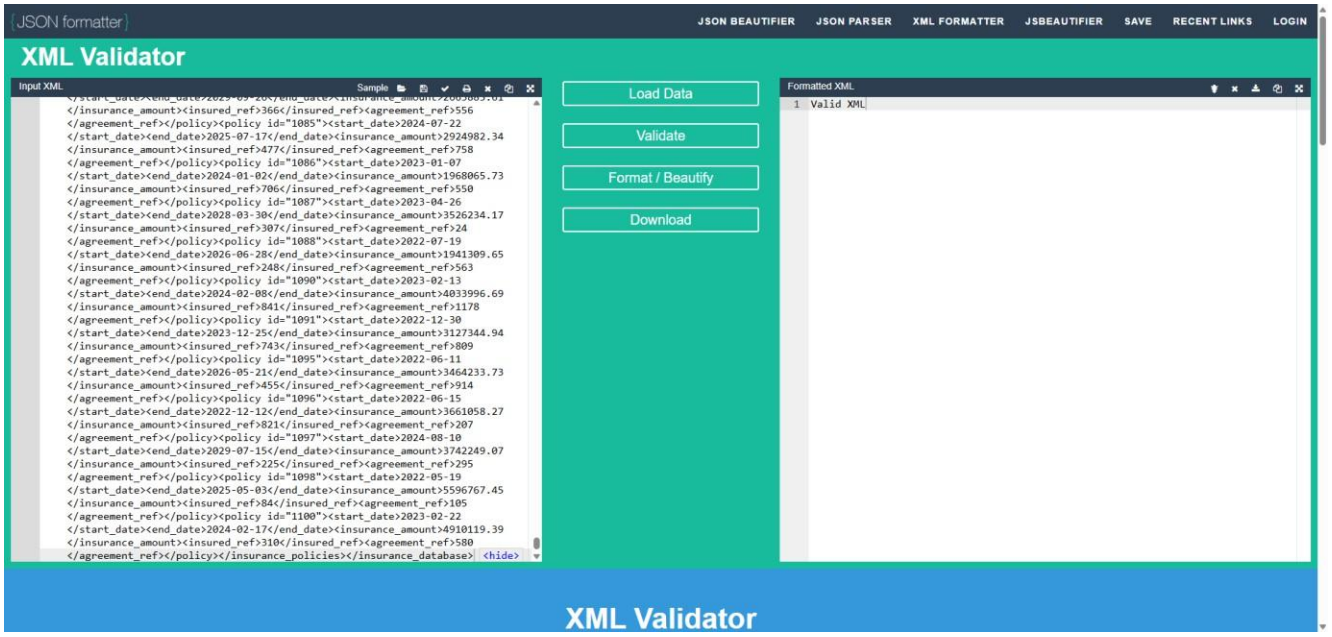
Data Output Messages Notifications

	xml_content text
1	<insurance_database export_date="2025-10-06T16:21:17.504943+03:00" system="Insurance System v1.0"><per

В результаті відформатував xml-контент, та отримав стандартну розмітку XML
Джерело - https://jsonformatter.org/xml-validator/



Використав синтаксичний валідатор для перевірки коректності створеного XML-документа



Відповіді на контрольні запитання

1. Що означає абревіатура XML?

XML розшифровується як eXtensible Markup Language (Розширювана мова розмітки).
Це мова розмітки, яка визначає набір правил для кодування документів у форматі, який зручний для читання і створення документів і людиною, і машиною.

2. Які три терміни використовуються для опису вмісту документа XML?

Три основні терміни для опису частин XML-документа:

1. Тег (Tag)

- Текст між лівою кутовою дужкою < та правою кутовою дужкою >
- Бувають початкові теги (наприклад, <name>) та кінцеві теги (наприклад, </name>)

Приклад:

```
<first-name>Mary</first-name>
```

2. Елемент (Element)

- Складається з початкового тегу, кінцевого тегу і всього, що знаходиться між ними
- Може містити дочірні елементи, текст або бути порожнім

Приклад:

```
<name>
    <title>Mrs.</title>
    <first-name>Mary</first-name>
    <last-name>McGoon</last-name>
</name>
```

Тут <name> елемент містить три дочірні елементи.

3. Атрибут (Attribute)

- Пара ім'я="значення" всередині початкового тегу елемента
- Надає додаткову інформацію про елемент

Приклад:

```
<city state="NC">Anytown</city>
```

Тут state є атрибутом елемента <city>.

3. Що означає поняття «правильно сформований» (well formed) документ?

Правильно сформований документ — це XML-документ, який відповідає всім синтаксичним правилам XML.

Основні вимоги:

1. Один кореневий елемент — документ повинен мати лише один головний елемент
2. Правильне закриття тегів — непорожні елементи повинні мати початковий та кінцевий теги
3. Порожні елементи можуть позначатися закритим тегом: <IAmEmpty/>
4. Унікальність атрибутів — один елемент не може мати кілька атрибутів з однаковим іменем
5. Значення атрибутів у лапках — використовуються одинарні ' або подвійні " лапки
6. Правильна вкладеність — теги можуть бути вкладені, але не можуть перекриватися
7. Правильне кодування — документ має складатися з правильно закодованих дозволених символів Unicode (UTF-8 або UTF-16)

Важливо: Документ, що не є правильно сформованим, **не може називатися XML-документом**. Сумісний синтаксичний аналізатор не повинен обробляти такі документи.

4. Що означає поняття «коректний» (valid) документ?
Коректний документ — це XML-документ, який:

1. Є правильно сформованим (відповідає синтаксичним правилам XML)
2. Містить посилання на граматичні правила (DTD або XML Schema)
3. Повністю відповідає обмеженням, вказаним у цих правилах

Приклад: Документ може бути правильно сформованим (всі теги закриті), але не коректним (якщо за схемою елемент <age> має бути числом, а в документі є текст).

5. Для чого в PostgreSQL при створенні XML-контенту використовується функція xmlagg?
xmlagg — це агрегатна функція, яка об'єднує (агрегує) XML-значення з кількох рядків в один XML-документ.
Призначення:

- Збирає результати з кількох рядків таблиці в єдину XML-структуру
- Працює аналогічно до інших агрегатних функцій (SUM, COUNT)
- Використовується у SELECT-запитах для створення списків елементів

```
SELECT xmlagg(
    xmlelement(NAME "person",
        xmlattributes(PERSON_ID AS "id"),
        xmlforest(FIRST_NAME AS "first_name", LAST_NAME AS "last_name")
    )
)
FROM PERSON_CHARACTERISTICS;
```

Результат:

```
<person id="1">
    <first_name>John</first_name>
    <last_name>Doe</last_name>
</person>
<person id="2">
    <first_name>Jane</first_name>
    <last_name>Smith</last_name>
</person>
```

Без xmlagg ми отримали б окремий XML-фрагмент для кожного рядка. З xmlagg — всі рядки об'єднуються в один XML-документ.

6. Для чого в PostgreSQL при створенні XML-контенту використовується функція xmlforest?
xmlforest — це функція, яка створює групу XML-елементів з набору стовпців або виразів.
Призначення:

- Спрощує створення кількох XML-елементів одночасно
- Автоматично перетворює кожен вираз у XML-елемент
- Зменшує кількість вкладених викликів xmlelement

Переваги: Код стає коротшим, читабельнішим і легшим для підтримки.

7. Що таке простір імен XML і навіщо в його оголошенні вказується URI?

Що таке простір імен (XML Namespace)?

Простір імен XML — це механізм для уникнення конфліктів імен елементів і атрибутів у XML-документах, коли використовуються елементи з різних словників (vocabularies).

Проблема:

Якщо два різні додатки використовують елемент <table>, це може означати:

- HTML: таблиця для відображення даних
- Меблева компанія: стіл як предмет меблів

Рішення — простори імен:

```
<document xmlns:html="http://www.w3.org/1999/xhtml"
          xmlns:furniture="http://example.com/furniture">

    <html:table>
        <html:tr>
            <html:td>Комірка таблиці</html:td>
        </html:tr>
    </html:table>

    <furniture:table>
        <furniture:width>120</furniture:width>
        <furniture:material>Дерево</furniture:material>
    </furniture:table>

</document>
```

Навіщо вказується URI?

URI (Uniform Resource Identifier) у просторі імен виконує роль унікального ідентифікатора, а НЕ посилання на веб-сторінку.

Причини використання URI:

1. Гарантія унікальності — доменні імена належать організаціям, тому конфлікти неможливі
2. Стандартизація — міжнародний стандарт для ідентифікації
3. Не обов'язково існує як веб-ресурс — це просто унікальний рядок

Приклад:

```
<insurance xmlns="http://university.edu/insurance/2025">
    <policy id="123"/>
</insurance>
```

URI *http://university.edu/insurance/2025* є просто унікальним ідентифікатором простору імен. Браузер не намагається його відкрити.

Висновок: на цій лабораторній роботі я ознайомився з конструкціями мови XML та її використанням в СКБД PostgreSQL, зокрема, зі створенням XML даних та XML документів, перетворенням таблиць реляційної бази в XML- документ.